

## ▼ Built-in PCA

```
import numpy as np
from sklearn.decomposition import PCA

# Original 2D dataset
X = np.asarray([[2, 1], [3, 5], [4, 3], [5, 6], [6, 7], [7, 8]]).astype(float)

# PCA with 1 component
pca = PCA(n_components=1)
X_pca = pca.fit_transform(X)

# Obtain the eigenmatrix (matrix of eigenvectors)
eigenmatrix = pca.components_

print("Eigenmatrix (Matrix of Eigenvectors):")
print(eigenmatrix)

# Recover the data back to the original space
X_recovered = pca.inverse_transform(X_pca)

# Print the explained variance ratio and the recovered data
print("Explained Variance Ratio:", pca.explained_variance_ratio_)
print("PCA-transformed Data:\n", X_pca)
print("Recovered Data:\n", X_recovered)
```

```
➦ Eigenmatrix (Matrix of Eigenvectors):
[[-0.56959484 -0.82192562]]
Explained Variance Ratio: [0.95623321]
PCA-transformed Data:
[[ 4.71168956]
 [ 0.85439226]
 [ 1.92864865]
 [-1.10672304]
 [-2.49824349]
 [-3.88976395]]
Recovered Data:
[[1.81624595 1.12734164]
 [4.01334258 4.29775312]
 [3.40145168 3.41479426]
 [5.13038373 5.90964402]
 [5.9229866  7.05337032]
 [6.71558946 8.19709663]]
```

## ▼ Our Defined PCA

```
def myPCA(X , num_components):

    print("\nMean Values :\t",np.mean(X , axis = 0))

    #Step-1
    means = np.mean(X , axis = 0)
    X_meaned = X - np.mean(X , axis = 0)
    X_meaned = np.round(X_meaned,2)
    print("Mean Value : \n", X_meaned)
    print("Shape", np.shape(means))

    #Step-2
    cov_mat = np.cov(X_meaned , rowvar = False)
    print("\n\nCovariance Matrix\n\n", cov_mat)

    #Step-3
    eigen_values , eigen_vectors = np.linalg.eigh(cov_mat)
    eigen_values = np.round(eigen_values,2)
    eigen_vectors = np.round(eigen_vectors,2)
    print("\nEigen Value : ", eigen_values)
    print("\nEigen Vector : \n\n", eigen_vectors)

    #Step-4
    sorted_index = np.argsort(eigen_values)[::-1]
    sorted_eigenvalue = eigen_values[sorted_index]
    sorted_eigenvectors = eigen_vectors[:,sorted_index]

    #Step-5
    eigenvector_subset = sorted_eigenvectors[:,0:num_components]
    print("\n\n",eigenvector_subset)
```

```
#Step-6
X_reduced = np.dot(eigenvector_subset.transpose() , X_meaned.transpose() ).transpose()
print("\n",X_reduced)

#Step 7
X_ret = np.dot(X_reduced, eigenvector_subset.transpose())
X_ret += means
print("\n",X_ret)
```

```
myPCA(X,1)
```

```
Mean Values :    [4.5  5. ]
Mean Value :
[[-2.5 -4. ]
 [-1.5  0. ]
 [-0.5 -2. ]
 [ 0.5  1. ]
 [ 1.5  2. ]
 [ 2.5  3. ]]
Shape (2,)
```

```
Covariance Matrix
```

```
[[3.5 4.4]
 [4.4 6.8]]
```

```
Eigen Value :  [0.45 9.85]
```

```
Eigen Vector :
```

```
[[-0.82  0.57]
 [ 0.57  0.82]]
```

```
[[0.57]
 [0.82]]
```

```
[[-4.705]
 [-0.855]
 [-1.925]
 [ 1.105]
 [ 2.495]
 [ 3.885]]
```

```
[[1.81815 1.1419 ]
 [4.01265 4.2989 ]
 [3.40275 3.4215 ]
 [5.12985 5.9061 ]
 [5.92215 7.0459 ]
 [6.71445 8.1857 ]]
```